

Step-by-Step Redux Integration Guide

Project: Employee_Project | Date: July 2026

Complete Step-by-Step Redux Integration Guide

Project: Employee_Project
Author: AI Assistant (Antigravity)
Date: July 2026

1. Introduction: Redux Concept in Simple Words

Redux is a state management tool that serves as a **Central Database** for your application.

- **Redux Store** (`store.js`): Think of this as the main warehouse. All global data (countries, states, languages, etc.) is stored here.
 - **Redux Slice** (`countriesSlice.js`): This is a department manager in the warehouse. It handles state initialization and defines instructions on how to update that specific data.
 - **Actions**: Instructions/signals sent from UI components telling Redux how to update state (e.g., "Add a country", "Delete a country").
 - **useSelector**: A React hook used in components to **read** data from the Redux store.
 - **useDispatch**: A React hook used in components to **send/trigger** actions.
-

2. Installation

To start using Redux in this project, install the following packages:

```
npm install @reduxjs/toolkit react-redux
```

3. Step 1: Create the Redux Slice

Create `src/redux/countriesSlice.js` to manage the list of countries.

```
import { createSlice } from '@reduxjs/toolkit';
const countriesSlice = createSlice({
  name: 'countries',
  initialState: {
    list: [], // Holds array of countries
  },
  reducers: {
    // Sets the list of countries (e.g., after fetching from API)
    setCountries: (state, action) => {
      state.list = action.payload;
    },
    // Adds a new country to the list
    addCountryState: (state, action) => {
      state.list.push(action.payload);
    },
    // Edits an existing country by matching its ID
    editCountryState: (state, action) => {
      const { id, name } = action.payload;
      const country = state.list.find((c) => c.id === id);
      if (country) {
        country.name = name;
      }
    }
  }
});
```

```

    }
  },
  // Deletes a country from the list
  deleteCountryState: (state, action) => {
    state.list = state.list.filter((c) => c.id !== action.payload);
  },
},
});
export const {
  setCountries,
  addCountryState,
  editCountryState,
  deleteCountryState,
} = countriesSlice.actions;
export default countriesSlice.reducer;

```

4. Step 2: Configure the Store

Create `src/redux/store.js` to combine your reducers and initialize the store.

```

import { configureStore } from '@reduxjs/toolkit';
import countriesReducer from './countriesSlice';
export const store = configureStore({
  reducer: {
    countries: countriesReducer, // register the slice here
  },
});
// Debugging Helper: Logs state changes to the browser console automatically
store.subscribe(() => {
  console.log('ðŸ”” Redux State Updated:', store.getState());
});

```

5. Step 3: Wrap the App with Redux Provider

Modify `src/main.jsx` to wrap your React App component inside the `` component.

```

import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux'; // <-- Import Provider
import { store } from './redux/store'; // <-- Import Store
import './index.css';
import App from './App.jsx';
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}> { /* <-- Wrap with Provider */}
    <App />
  </Provider>
</StrictMode>,
);

```

6. Step 4: Use Redux in Component (CountryPage)

Modify `src/pages/CountryPage.jsx` to fetch and dispatch actions to the Redux store.

```

import React, { useState, useEffect } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import {
  setCountries,
  addCountryState,
  editCountryState,
  deleteCountryState
} from '../../redux/countriesSlice';
import api from '../../Api/axios';
const CountryPage = () => {

```

```

// Read state from Redux store
const countries = useSelector((state) => state.countries.list);
const dispatch = useDispatch();
const [countryName, setCountryName] = useState('');
const [editId, setEditId] = useState(null);
// Fetch countries on mount
useEffect(() => {
  const fetchCountries = async () => {
    try {
      const response = await api.get('/Country');
      const mapped = response.data.map((c) => ({
        id: c.id,
        name: c.countryName
      }));
      dispatch(setCountries(mapped)); // Dispatch action to store data
    } catch (error) {
      console.error('Error fetching countries:', error);
    }
  };
  fetchCountries();
}, [dispatch]);
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!countryName.trim()) return;
  if (editId !== null) {
    try {
      await api.put(`/Country/${Number(editId)}`, { Id: Number(editId), CountryName:
countryName.trim() });
      dispatch(editCountryState({ id: editId, name: countryName.trim() }));
      setEditId(null);
    } catch (error) {
      alert('Error editing country: ' + error.message);
    }
  } else {
    try {
      const response = await api.post('/Country', { CountryName: countryName.trim() });
      const newCountry = {
        id: response.data.id || response.data.Id,
        name: response.data.countryName || response.data.CountryName
      };
      dispatch(addCountryState(newCountry));
    } catch (error) {
      alert('Error adding country: ' + error.message);
    }
  }
  setCountryName('');
};
const handleDeleteClick = async (id) => {
  try {
    await api.delete(`/Country/${Number(id)}`);
    dispatch(deleteCountryState(id));
  } catch (error) {
    alert('Error deleting country: ' + error.message);
  }
};
// ... rest of CountryPage UI rendering logic using 'countries' array
};

```

7. How to Verify State Updates

1. Open your browser and go to your app (`http://localhost:5174/`).
2. Press **F12** and select the **Console** tab.
3. Perform any Add/Edit/Delete actions on countries.
4. You will see console logs starting with `Redux State Updated:` displaying the new state in real-time.
5. Alternatively, download the **Redux DevTools** browser extension to view visual state timelines, state trees, and diffs.

